Tu fais face à une **CORS error**, et c'est parfaitement logique dans ta nouvelle configuration :
Ton frontend tourne sur **http://localhost**
Ton backend tourne sur **http://api.localhost**

➡️ **Ce sont deux origines différentes**, donc le navigateur bloque la requête.
C'est exactement ce que décrit la documentation CORS : le serveur doit explicitement autoriser l'origine du frontend [1].

Je vais t'expliquer **pourquoi ça arrive**, **comment ça marche**, et surtout **comment corriger proprement dans ton API PHP**.

🔥 Pourquoi tu as une CORS error ?

Le navigateur applique la **Same-Origin Policy** :
Deux origines sont différentes si **protocole, domaine ou port** diffèrent.

Frontend	Backend	Origines différentes ?
http://localhost	http://api.localhost	✅ Oui

Donc le navigateur envoie d'abord une requête **OPTIONS (preflight)** pour demander :

- "Est-ce que j'ai le droit d'appeler ton API ?"
- "Est-ce que tu acceptes JSON ?"
- "Est-ce que tu acceptes POST/PUT/DELETE ?"

Si ton backend **ne répond pas correctement**, le navigateur bloque la requête.
C'est exactement ce que décrit la documentation CORS : le serveur doit renvoyer les bons headers, notamment `Access-Control-Allow-Origin` et `Access-Control-Allow-Headers` [2].

✅ La solution PRO : ajouter les bons headers CORS dans ton API

Dans ton fichier **backend/public/index.php**, ajoute **tout en haut** :

```
// CORS
header("Access-Control-Allow-Origin: http://localhost");
header("Access-Control-Allow-Credentials: true");
```

```
header("Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS");
header("Access-Control-Allow-Headers: Content-Type, Authorization");

// Réponse immédiate au preflight OPTIONS
if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(200);
    exit();
}
```

Pourquoi ça marche ?

- `Access-Control-Allow-Origin` → autorise ton frontend
- `Allow-Methods` → autorise POST/PUT/DELETE
- `Allow-Headers` → autorise JSON, tokens, etc.
- `OPTIONS` → répond au preflight comme recommandé par les bonnes pratiques CORS [1]

⚠️ Important : n'utilise pas `*` avec credentials

Si ton frontend utilise :

- `fetch(..., { credentials: "include" })`
- ou des cookies

Alors `Access-Control-Allow-Origin: *` est **interdit** par la spec CORS.

🧪 Vérification rapide

1. Ouvre l'onglet Network → clique sur la requête OPTIONS
2. Vérifie que la réponse contient :

```
 Access-Control-Allow-Origin: http://localhost
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type
```

3. Recharge → la CORS error disparaît.

🎯 Résumé

Tu avais une CORS error parce que :

- ton frontend et backend sont sur deux origines différentes
- ton backend ne renvoyait pas les headers CORS nécessaires
- le navigateur bloquait la requête avant même qu'elle n'arrive à PHP

Maintenant, avec les headers ci-dessus, ton API REST accepte proprement les requêtes cross-origin.

Si tu veux, je peux aussi te générer :

- une **classe CORS propre** (middleware)
- une version **configurable** (liste d'origines autorisées)
- une version **production-ready** (sécurisée, restrictive)

Tu veux laquelle ?

References (2)

1 *Cross-Origin Request Headers (CORS) with PHP headers.* https://stackoverflow.com/questions/8719276/cross-origin-request-headerscors-with-php-headers

2 *PHP CORS Headers: How to Fix "Request Header Field X-Requested-With Not ....* https://www.xjavascript.com/blog/cross-origin-request-headers-cors-with-php-headers/